6. 산술 연산 및 필드 연산 모듈 분석

개요

Circom 컴파일러에서 산술 연산과 필드 연산은 zkSNARK 회로의 핵심을 이루는 부분입니다. 이 문서는 `circom_algebra` 모듈의 필드 연산 구현과 `constant_tracking` 모듈의 상수 최적화 기법을 상세히 분석합니다. 모든 연산은 유한체(Finite Field) 위에서 수행되며, R1CS 제약 조건 생성을 위한 산술 표현식 처리가 핵심 기능입니다.

6.1 circom_algebra 모듈의 필드 연산 분석

ArithmeticExpression 데이터 구조

`circom_algebra/src/algebra.rs` 에서 정의된 `ArithmeticExpression` 은 Circom 회로의 모든 산술 표현식을 표현하는 핵심 데이터 구조입니다:

```
pub enum ArithmeticExpression<C>
where
C: Hash + Eq,
{
Number {
value: BigInt,
},
Signal {
symbol: C,
```

```
},
Linear {
// c1*s1 + .. + cn*sn + C 형태의 선형 표현식
coefficients: HashMap<C, BigInt>,
},
Quadratic {
// a*b + c 형태의 이차 표현식
// a, b, c는 모두 선형 표현식
a: HashMap<C, BigInt>,
b: HashMap<C, BigInt>,
c: HashMap<C, BigInt>,
},
NonQuadratic,
}
```

표현식 계층 구조

1. **Number**: 상수 값 (BigInt)
2. **Signal**: 단일 신호 변수
3. **Linear**: 선형 조합 ($c_1s_1 + c_2s_2 + \ldots + \text{constant}$)
4. **Quadratic**: 이차 표현식 ($(a_1s_1 + a_2s_2 + \ldots) * (b_1s_1 + b_2s_2 + \ldots) + (c_1s_1 + c_2s_2 + \ldots)$)
5. **NonQuadratic**: R1CS로 표현 불가능한 고차 표현식

유한체 연산 구현 (`modular_arithmetic.rs`)

기본 산술 연산

```
// 모듈러 덧셈
pub fn add(left: &BigInt, right: &BigInt, field: &BigInt) -> BigInt {
modulus(&(left + right), field)
}

// 모듈러 곱셈
pub fn mul(left: &BigInt, right: &BigInt, field: &BigInt) -> BigInt {
modulus(&(left * right), field)
}

// 모듈러 나눗셈 (역원 계산 포함)
pub fn div(left: &BigInt, right: &BigInt, field: &BigInt) -> Result<BigInt, Ar
let right_inverse = right
.mod_inverse(field)
.map_or(Result::Err(ArithmeticError::DivisionByZero), |a| Result::Ok(a))?;
let res = mul(left, &right_inverse, field);
Result::Ok(res)
}
```

최적화된 다중 역원 계산

```
pub fn multi_inv(values: &Vec<BigInt>, field: &BigInt) -> Vec<BigInt> {
let one: BigInt = BigInt::from(1);
```

```
let mut partials: Vec<BigInt> = Vec::new();
partials.push(one.clone());
// 부분 곱 계산: [1, a₁ , a₁ a₂ , a₁ a₂ a₃ , ...]
for i in 0..values.len() {
partials.push(mul(partials.get(partials.len()-1).unwrap(),
values.get(i).unwrap(),
&field));
}
// 전체 곱의 역원 계산
let mut inverse = div(&one,
partials.get(partials.len()-1).unwrap(),
&field).ok().unwrap();
// 역방향으로 개별 역원들 계산
let mut outputs: Vec<BigInt> = vec![BigInt::from(0); partials.len()];
let mut i = values.len();
while i > 0 {
outputs[i-1] = mul(partials.get(i-1).unwrap(), &inverse, &field);
inverse = mul(&inverse, values.get(i-1).unwrap(), &field);
i = i - 1;
}
return outputs;
}
```

최적화 효과: 개별적으로 n개의 역원을 계산하면 O(n log p) 시간이 걸리지만, 이 방법은 O(log p + n) 시간에 모든 역원을 계산할 수 있습니다.

ArithmeticError 처리 방법

```
pub enum ArithmeticError {
DivisionByZero, // 0으로 나누기 시도
BitOverFlowInShift, // 비트 시프트 오버플로우
}
```

에러 처리는 Result 타입을 통해 안전하게 수행되며, 특히 나눗셈과 비트 연산에서 중요합니다.

산술 표현식 연산 구현

덧셈 연산 (add 함수)

```
pub fn add(
left: &ArithmeticExpression<C>,
right: &ArithmeticExpression<C>,
field: &BigInt,
) -> ArithmeticExpression<C> {
use ArithmeticExpression::*;
match (left, right) {
// 이차식 + 이차식 = 비이차식 (R1CS 한계)
(Quadratic { .. }, Quadratic { .. }) => NonQuadratic,
// 상수 + 상수 = 상수
```

```
(Number { value: v_0 }, Number { value: v_1 }) => {
Number { value: modular_arithmetic::add(v_0, v_1, field) }
}
// 상수 + 신호 = 선형식
(Number { value }, Signal { symbol }) | (Signal { symbol }, Number { value }) =>
let mut coefficients = HashMap::new();
ArithmeticExpression::initialize_hashmap_for_expression(&mut coefficients);
ArithmeticExpression::add_constant_to_coefficients(value, &mut coefficients, fi
ArithmeticExpression::add_symbol_to_coefficients(
symbol, &BigInt::from(1), &mut coefficients, field,
);
Linear { coefficients }
}
// 추가 케이스들...
}
}
```

곱셈 연산 (`mul` 함수)

곱셈은 R1CS의 핵심 제약사항을 반영합니다:

```
pub fn mul(
left: &ArithmeticExpression<C>,
right: &ArithmeticExpression<C>,
field: &BigInt,
```

```
) -> ArithmeticExpression<C> {
use ArithmeticExpression::*;
match (left, right) {
// 이차식과 다른 식의 곱 = 비이차식
(Quadratic { .. }, _) | (_, Quadratic { .. }) => NonQuadratic,
// 선형식 * 선형식 = 이차식
(Linear { coefficients: coefficients_0 }, Linear { coefficients: coefficients_1
let a = coefficients_0.clone();
let b = coefficients_1.clone();
let mut c = HashMap::new();
ArithmeticExpression::initialize_hashmap_for_expression(&mut c);
Quadratic { a, b, c }
}
// 신호 * 신호 = 이차식
(Signal { symbol: symbol_0 }, Signal { symbol: symbol_1 }) => {
let mut a = HashMap::new();
let mut b = HashMap::new();
let mut c = HashMap::new();
// a, b, c 초기화 및 계수 설정
Quadratic { a, b, c }
}
}
```

```
}
```

비트 연산 및 불린 연산

비트 연산

```
pub fn bit_and(left: &BigInt, right: &BigInt, field: &BigInt) -> BigInt {
modulus(&(left & right), field)
}

pub fn shift_l(left: &BigInt, right: &BigInt, field: &BigInt) -> Result<BigInt
let two = BigInt::from(2);
let top = field / &two;
if right <= &top {
let usize_repr = right.to_usize()
.map_or(Result::Err(ArithmeticError::DivisionByZero), |a| Result::Ok(a))?;
let value = modulus(&((left * &num_traits::pow(two, usize_repr)) & &mask(field)
Result::Ok(value)
} else {
shift_r(left, &(field - right), field)
}
}
```

불린 연산

```
pub fn bool_and(left: &BigInt, right: &BigInt, field: &BigInt) -> BigInt {
normalize(left, field) * normalize(right, field)
}

pub fn bool_or(left: &BigInt, right: &BigInt, field: &BigInt) -> BigInt {
(normalize(left, field) + normalize(right, field) + bool_and(left, right, field)
}
```

R1CS 제약 조건 변환

```
pub fn transform_expression_to_constraint_form(
arithmetic_expression: ArithmeticExpression<C>,
field: &BigInt,
) -> Option<Constraint<C>> {
use ArithmeticExpression::*;
let mut a = HashMap::new();
let mut b = HashMap::new();
let mut c = HashMap::new();
match arithmetic_expression {
NonQuadratic => Option::None,
Quadratic { a: old_a, b: old_b, c: old_c } => {
a = old_a;
```

```
b = old_b;

c = old_c;

}

// 다른 타입들을 이차식으로 변환

Number { value } => {

c.insert(ArithmeticExpression::constant_coefficient(), value);

}

// ...

}

// c를 음수로 변환 (A * B = C 형태로 만들기 위해)

ArithmeticExpression::multiply_coefficients_by_constant(&BigInt::from(-1), &mut

Option::Some(Constraint::new(a, b, c))

}
```

다항식 연산 및 계수 처리

계수 관리 시스템

모든 HashMap은 상수 계수를 위한 기본 키를 포함해야 합니다:

```
fn  initialize_hashmap_for_expression(initial: &mut HashMap<C, BigInt>) {

initial

.entry(ArithmeticExpression::constant_coefficient())

.or_insert_with(|| BigInt::from(0));

}
```

```
fn add_symbol_to_coefficients(
symbol: &C,
coefficient: &BigInt,
coefficients: &mut HashMap<C, BigInt>,
field: &BigInt,
) {
ArithmeticExpression::initialize_symbol_in_coefficients(symbol, coefficients);
coefficients.insert(
symbol.clone(),
modular_arithmetic::add(coefficients.get(symbol).unwrap(), coefficient, field),
);
}
```

계수 정규화

```
fn multiply_coefficients_by_constant(
constant: &BigInt,
coefficients: &mut HashMap<C, BigInt>,
field: &BigInt,
) {
for value in coefficients.values_mut() {
*value = modular_arithmetic::mul(value, constant, field);
```

```
}
}

fn divide_coefficients_by_constant(
constant: &BigInt,
coefficients: &mut HashMap<C, BigInt>,
field: &BigInt,
) -> Result<(), ArithmeticError> {
let inverse_constant = modular_arithmetic::div(&BigInt::from(1), constant, &fie
ArithmeticExpression::multiply_coefficients_by_constant(&inverse_constant, coeff
Result::Ok(())
}
```

최적화 기법

1. 제곱 연산 최적화

```
pub fn pow(
left: &ArithmeticExpression<C>,
right: &ArithmeticExpression<C>,
field: &BigInt,
) -> ArithmeticExpression<C> {
match (left, right) {
// x^2 최적화: 별도의 곱셈 제약 조건 대신 직접 처리
```

```
(Signal { symbol }, Number { value }) if *value == BigInt::from(2) => {
let  left = Signal { symbol: symbol.clone() };
let  right = Signal { symbol: symbol.clone() };
ArithmeticExpression::mul(&left, &right, field)
}
// 선형식의 제곱도 동일하게 처리
(Linear { coefficients }, Number {value}) if *value == BigInt::from(2) => {
let  left = Linear { coefficients: coefficients.clone() };
let  right = Linear { coefficients: coefficients.clone() };
ArithmeticExpression::mul(&left, &right, field)
}
_ => NonQuadratic,
}
}
```

2. 메모리 효율성

- HashMap 사용으로 희소 다항식 효율적 표현
- 0 계수 자동 제거
- 상수 계수 특별 처리

3. 타입 안전성

- 제네릭 타입 C를 통한 신호 식별자 추상화
- Result 타입을 통한 안전한 에러 처리

- 컴파일 타임 타입 검사

이러한 구현을 통해 Circom 컴파일러는 복잡한 산술 표현식을 효율적으로 R1CS 제약 조건으로 변환할 수 있으며, 유한체 연산의 특성을 완전히 활용합니다.

6.2

상수 추적 및 최적화 분석

ConstantTracker 구조 및 역할

`constant_tracking/src/lib.rs` 에서 정의된 `ConstantTracker` 는 컴파일 과정에서 상수 값들을 효율적으로 관리하는 핵심 구조체입니다:

```
pub struct ConstantTracker<C>
where
C: Hash,
{
lookup: HashMap<C, CID>, // 상수 값 → ID 매핑
constants: Vec<C>, // ID → 상수 값 매핑
}
```

주요 기능

1. **상수 중복 제거**: 동일한 상수 값에 대해 하나의 ID만 할당
2. **메모리 효율성**: 상수 값을 한 번만 저장하고 ID로 참조
3. **빠른 조회**: HashMap을 통한 O(1) 상수 검색

```
impl<C> ConstantTracker<C>
```

```
where
C: Eq + Hash + Clone,
{
pub fn insert(&mut self, constant: C) -> CID {
if let Some(id) = self.get_id(&constant) {
id // 이미 존재하는 상수는 기존 ID 반환
} else {
let id = self.constants.len();
self.constants.push(constant.clone());
self.lookup.insert(constant, id);
id // 새로운 상수는 새 ID 할당
}
}

pub fn get_constant(&self, id: CID) -> Option<&C> {
if id < self.constants.len() {
Some(&self.constants[id])
} else {
None
}
}
}
```

제약 조건 압축 저장 시스템

`circom_algebra/src/constraint_storage/` 에서 구현된 압축 저장 시스템은 `ConstantTracker` 를 활용하여 메모리 사용량을 최적화합니다:

```
type CompressedExpr = Vec<(CID, S)>; // (계수_ID, 신호) 쌍들

type CompressedConstraint = (CompressedExpr, CompressedExpr, CompressedExpr); //


pub struct ConstraintStorage {

field_tracker: FieldTracker, // BigInt 계수들을 추적

constraints: Vec<CompressedConstraint>, // 압축된 제약 조건들

}
```

압축 과정

```
pub fn code_expression(expr: HashMap<S, BigInt>, tracker: &mut FieldTracker) -

let mut c_expr = CompressedExpr::new();

for (var, coeff) in expr {

let raw_coeff = coeff.to_signed_bytes_le(); // BigInt를 바이트 배열로 변환

let coeff_id = tracker.insert(raw_coeff); // 상수 추적기에 저장

c_expr.push((coeff_id, var));

}

c_expr

}
```

메모리 절약 효과: 동일한 계수 값(예: 1, -1, 0)이 여러 제약 조건에서 반복 사용될 때, 실제 BigInt 값은 한 번만 저장되고 ID로 참조됩니다.

제약 조건 단순화 알고리즘

`constraint_list/src/constraint_simplification.rs` 에서 구현된 단순화 과정은 여러 단계의 최적화를 수행합니다:

1. 클러스터 기반 병렬 처리

```
fn build_clusters(linear: LinkedList<C>, no_vars: usize) -> Vec<Cluster> {
let mut signal_to_cluster = vec![no_linear; no_vars];
for constraint in linear {
let signals = C::take_cloned_signals(&constraint);
let dest = ClusterArena::len(&arena);
// 제약 조건을 클러스터에 추가
ClusterArena::push(&mut arena, Some(Cluster::new(constraint, signals.len())));
for signal in signals {
let prev = signal_to_cluster[signal];
signal_to_cluster[signal] = dest;
if prev < no_linear {
// 공통 신호를 가진 제약 조건들을 같은 클러스터로 병합
arena_merge(&mut arena, &mut cluster_to_current, prev, dest);
}
}
}
```

```
}
```

최적화 효과: 공통 신호를 공유하는 제약 조건들을 그룹화하여 병렬 처리하고, 신호 대체를 효율적으로 수행합니다.

2. 등식 제약 조건 단순화

```
fn eq_cluster_simplification(
mut cluster: Cluster,
forbidden: &HashSet<usize>,
field: &BigInt,
) -> (LinkedList<S>, LinkedList<C>) {
if Cluster::size(&cluster) == 1 {
// 단일 등식: s₀ = s₁ → s₁ 을 s₀로 대체
let constraint = LinkedList::pop_back(&mut cluster.constraints).unwrap();
let signals: Vec<_> = C::take_cloned_signals_ordered(&constraint).iter().cloned
let s_0 = signals[0];
let s_1 = signals[1];
if HashSet::contains(forbidden, &s_0) && HashSet::contains(forbidden, &s_1) {
// 둘 다 금지된 신호면 제약 조건 유지
LinkedList::push_back(&mut constraints, constraint);
} else {
// 하나를 다른 것으로 대체
let (l, r) = if s_0 > s_1 { (s_0, s_1) } else { (s_1, s_0) };
LinkedList::push_back(&mut substitutions, S::new(l, A::Signal { symbol: r }).un
```

```
}
}
}
```

3. 상수 등식 처리

```
fn constant_eq_simplification(
c_eq: LinkedList<C>,
forbidden: &HashSet<usize>,
field: &BigInt,
) -> (LinkedList<S>, LinkedList<C>) {
for constraint in c_eq {
// 상수 등식: signal = constant
let signal = signals.pop().unwrap();
if !HashSet::contains(&forbidden, &signal) {
// signal을 상수 값으로 대체
let sub = C::clear_signal_from_linear(constraint, &signal, field);
LinkedList::push_back(&mut subs, sub);
}
}
}
```

1. 다중 역원 계산 최적화

circom_algebra/src/simplification_utils.rs 에서 구현된 정규화 과정:

```
fn normalize_substitutions(substitutions: SHNotNormalized, field: &BigInt) -> S
let mut coeffs: Vec<BigInt> = Vec::new();
// 모든 계수들을 수집
for (_signal, (coeff, _sub)) in &substitutions {
coeffs.push(coeff.clone());
}
// 한 번에 모든 역원 계산 (O(log p + n) 시간)
let inverses = modular_arithmetic::multi_inv(&coeffs, field);
// 정규화된 대체 규칙 생성
let mut i = 0;
for (signal, (_coeff, sub)) in substitutions {
let inv = inverses.get(i).unwrap();
let arith_sub = A::hashmap_into_arith(sub.to().clone());
let mult_by_inverse = A::mul(&arith_sub, &A::Number {value: inv.clone()}, field
let new_sub = S::new(signal.clone(), mult_by_inverse).unwrap();
tree.insert(signal, new_sub);
i = i + 1;
}
}
```

2. 신호 선택 휴리스틱

```
fn take_signal_4(
signals: &SignalDefinition4,
info_ocurrences: &SignalsInformation,
constraint: &C
) -> Option<usize> {
let mut ret = Option::None;
let mut ocurrences_ret: Option<usize> = Option::None;
for k in constraint.c().keys() {
if signals.can_be_taken(*k) {
let new_ocurrences = info_ocurrences.signal_to_ocurrences.get(k).unwrap();
match ocurrences_ret {
Some(val_ant) => {
if *new_ocurrences < val_ant {
// 더 적게 사용되는 신호를 우선 선택
ret = Some(*k);
ocurrences_ret = Some(*new_ocurrences);
}
}
None => {
ret = Some(*k);
ocurrences_ret = Some(*new_ocurrences);
```

```
}
}
}
}
ret
}
```

최적화 원리: 사용 빈도가 낮은 신호를 먼저 제거하여 전체 제약 조건 수를 최소화합니다.

3. 동적 값과 정적 값 구분

컴파일 타임에 결정할 수 있는 값들을 구분하여 처리:

```
// 컴파일 타임 상수
pub enum ArithmeticExpression<C> {
Number { value: BigInt }, // 정적 상수
Signal { symbol: C }, // 동적 신호
Linear { coefficients: HashMap<C, BigInt> }, // 부분적으로 정적
// ...
}

// 상수 전파 최적화
pub fn add(left: &ArithmeticExpression<C>, right: &ArithmeticExpression<C>, fi
match (left, right) {
// 두 상수의 덧셈 → 컴파일 타임에 계산
(Number { value: v_0 }, Number { value: v_1 }) => {
```

```
Number { value: modular_arithmetic::add(v_0, v_1, field) }

}

// 상수 + 신호 → 선형식으로 변환

(Number { value }, Signal { symbol }) => {

// 계수 맵에 상수와 신호 추가

}

// ...

}

}
```

메모리 및 성능 최적화

1. 희소 표현 (Sparse Representation)

```
// 밀집 배열 대신 HashMap 사용으로 0이 아닌 계수만 저장

Linear { coefficients: HashMap<C, BigInt> }


// 0 계수 자동 제거

fn  remove_zero_value_coefficients(constraint: &mut C) {

constraint.a.retain(|_, v| !v.is_zero());

constraint.b.retain(|_, v| !v.is_zero());

constraint.c.retain(|_, v| !v.is_zero());

}
```

2. 병렬 처리

```
// ThreadPool을 사용한 클러스터 병렬 처리
let pool = ThreadPool::new(num_cpus::get());
for cluster in clusters {
let job = move || {
let result = eq_cluster_simplification(cluster, &forbidden, &field);
cluster_tx.send((id, result)).unwrap();
};
ThreadPool::execute(&pool, job);
}
```

3. 점진적 최적화

```
// 여러 라운드에 걸친 점진적 단순화
let mut apply_round = apply_linear && no_rounds > 0 && !linear.is_empty();
while apply_round {
let (substitutions, constants) = linear_simplification(/* ... */);
// 새로운 선형 제약 조건 생성
linear = apply_substitution_to_map(&mut constraint_storage, &mut non_linear_ma
no_rounds -= 1;
apply_round = !linear.is_empty() && no_rounds > 0;
}
```

최적화 효과 측정

1. **제약 조건 수 감소**: 신호 대체를 통해 전체 제약 조건 수 최소화
2. **메모리 사용량 감소**: 상수 중복 제거 및 압축 저장
3. **컴파일 시간 단축**: 병렬 처리 및 효율적인 알고리즘
4. **증명 생성 시간 단축**: 최적화된 회로로 인한 성능 향상

이러한 종합적인 최적화 기법들을 통해 Circom 컴파일러는 복잡한 회로도 효율적으로 처리할 수 있으며, 실제 zkSNARK 증명 시스템에서 요구되는 성능을 달성합니다.